Aidan Rohm - Final Project - Professor Lyons - 5.2.26

Probabilistic Terrain Traversability Estimation -- Report

1. Problem Statement

The goal of this project is to estimate the traversability of an enclosed terrain using a TurtleBot3 in Gazebo. Traversability is a hidden Boolean random variable that varies by location. Some regions of the floor are smooth, and the wheels grip normally, while others are slippery, and the wheels lack sufficient traction to control the robot's velocity. The robot can't directly observe traversability; it only observes wheel slip, which is influenced by the underlying terrain.

The resulting table is a 2D grid of P(traversable) values covering the enclosure, discretized into 0.5 x 0.5 m cells. The prior at each cell is <0.5, 0.5> with contiguous regions of the floor sharing the traversability properties.

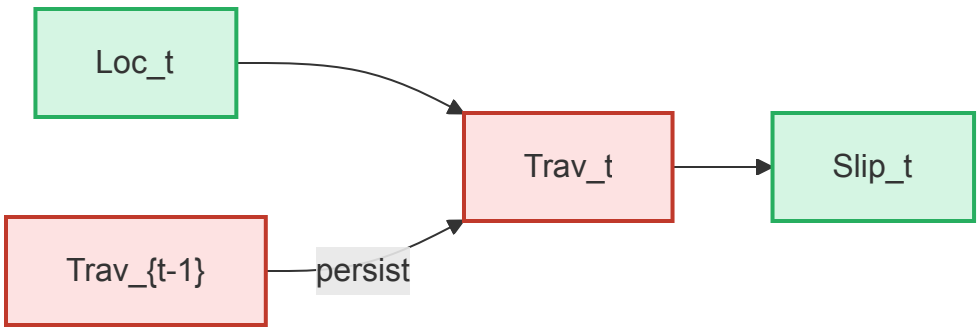
2. Dynamic Bayesian Network Design

2.1 Random Variables

I used three distinct random variables in the DBN:

- Location_t = (x,y): observed. The robot's pose at time t, taken from /odom and discretized into a grid cell
- Traversable_t: hidden Boolean. Whether the cell at Location_t has good grip (true) or is slippery (false)
- WheelSlip_t: observed continuous

2.2 Network Structure



There are three edges here:

- Location_t -> Traversable_t. The cell the robot is in selects which Traversable variable is being queried. This is a deterministic indexing relationship rather than a stochastic CPT. The

location tells us which of the per-cell Traversable beliefs need to be updated

- Traversable_t -> WheelSlip_t. This is the sensor model in which slippery terrain produces high slip and gripping terrain produces low slip. This is the edge where the CPT lives.
- Traversable*{t-1} -> Traversable_t. The temporal persistence edge. The assignment states the terrain is static, making this transition the identity: P(Traversable_t = True |* Traversable{t-1} = True) = 1. The implication is that every observation in a given cell contributes to the same belief. There is no decay, no transition noise, and no reason to forget old observations

2.3 Sensor Model (CPT for WheelSlip given Traversable)

WheelSlip is continuous, so I represented the CPT as a parametric likelihood function rather than a discrete table:

$$P(\text{Traversable} = \text{True} \mid \overline{\text{slip}}) = e^{-K \cdot \overline{\text{slip}}}$$

where

$$\overline{\text{slip}}$$

is the running mean of slip observations in that cell and K is a tunable parameter that controls how sharply slip discriminates between slippery and gripping terrain.

This general shape was chosen because:

- It is monotonically decreasing in slip; more slip always means lower P(traversable)
- It produces graded probabilities rather than slamming to 0 or 1, which preserves uncertainty for cells with only a few observations
- It is bounded in 0 and 1 for any non-negative slip value, which is necessary because slip is a non-negative quantity
- Setting K=20 gives a meaningful spread across slip values actually observed in this world:

Mean slip	P(Traversable = True)	Interpretation
0.000	1.000	perfect grip
0.020	0.670	mild slip
0.050	0.368	moderate slip
0.100	0.135	clearly slippery
0.200	0.018	very slippery

The choice of K is the single tunable parameter of the model. I tuned it by inspecting the slip values published on /terrain_features during a coverage run and choosing a value that gave a clear separation between the normal and slippery regions of the world.

State Estimation Algorithm

3.1 Filtering Approach

Because the temporal transition is the identity (terrain doesn't change), the filtering equation collapses dramatically. The general filtering recursion is:

$$P(X_t \mid e_{1:t}) \propto P(e_t \mid X_t) \sum_{x_{t-1}} P(X_t \mid x_{t-1}), P(x_{t-1} \mid e_{1:t-1})$$

With the static-terrain assumption, P(Xt∣xt−1)P(X*t* *mid x*{t-1}) P(Xt∣xt−1) is deterministic identity, so the recursion becomes:

$$P(\text{Traversable} \mid e_{1:t}) \propto P(e_t \mid \text{Traversable}) \cdot P(\text{Traversable} \mid e_{1:t-1})$$

This is the standard Bayesian update for a static parameter with a stream of conditionally independent observations. For static terrain, this is mathematically equivalent to maintaining sufficient statistics over all slip observations in a cell and computing the posterior in closed form at any time. I chose the latter implementation because it is numerically more stable than repeated multiplicative Bayesian updates (which can drift toward 0 or 1 after many observations and lose precision).

3.2 Per-Cell Implementation

For each grid cell (r, c) there are two accumulators:

- slip_sum: the sum of all slip observations recorded in that cell
- slip_count: the number of observations recorded in that cell
  When a feature message arrives on /terrain_features with a slip value and pose:
- Convert (x, y) to grid coordinates using the world_to_grid function
- If the cell is within bounds, accumulate the slip and increment the count using the above structure
  To produce the posterior at any time:
- Compute the per-cell mean:

$$\overline{\text{slip}}[r, c] = \frac{\text{slip\_sum}[r, c]}{\text{slip\_count}[r, c]}$$

- Apply the likelihood:

$$P(\text{Traversable} = \text{True}) = e^{-K \cdot \overline{\text{slip}}[r,c]}$$

- For unvisited cells, retain the specified prior of 0.5

3.3 Justification for Mean-Slip Aggregation

The slip signal is noisy by default. Each individual instance of a slip reading reflects not only the underlying terrain but also momentary acceleration, control loop lag, and sensor noise. A single observation of low or high slip does not necessarily mean that any individual cell is slippery. Averaging multiple observations within a cell directly addresses this.

4. Map Generation Design

4.1 Discretization

The world frame extends from $x \in [-3.5, 3.5]$ and $y \in [-5.0, 1.7]$ which I divided into 0.5 x 0.5m cells. This yields a grid of 13 rows and 14 columns. The transformation from world coordinates to grid indices is:
col = floor((x − x_min) / cell_size)
row = floor((y − y_min) / cell_size)
With row 0 corresponding to the lowest y. The grid origin is the bottom left corner of the enclosure, which keeps the row/column convention aligned with image coordinates when displayed with origin="lower" in matplotlib.

4.2 Robot Coverage Strategy

Attempt 1:

- Took the form of a lawnmower sweep
- Vertical strips spaced 0.45m apart traversed in alternating directions
- This would have worked well in theory, but because of the slippage and enclosure structure, the robot had issues staying away from obstacles
  Attempt 2:
- Hard coded a tour of 20 waypoints, 5 within each of the four colored quadrants
- Each waypoint sits well inside its quadrant and away from the walls and inner obstacle
- The robot drives from waypoint to waypoint using features to prevent it from getting stuck or stalled on one point for more than 25 seconds, trying to reach an obscured waypoint more than three times, backup and turn recovery, and finally a clean shutdown
- The waypoints deliberately avoid the protruding obstacle in the middle of the enclosure

4.3 ROS Node Architecture

Three nodes are running concurrently:

- rohm_finalproj_featureextract.py which publishes a JSON message containing slip and pose to /terrain_features at 5 HZ
- rohm_finalproj_coverage_mover.py which drives the robot through the 20-waypoint tour, publishing velocity commands to /cmd_vel

- rohm_finalproj_traversability_estimator.py which subscribes to /terrain_features and accumulates slip observations per cell, prints the posterior matrix every 5 seconds, and saves a heatmap PNG
  By decoupling the feature extraction from estimation, the sensor model is separated from the inference which mirrors the structure of the DBN itself. The feature node materializes the WheelSlip evidence, and the estimator does the filtering on Traversable.

4.4 Visualization

The estimator produces two outputs every 5 seconds

- Text matrix printed to stdout with rows ordered by descending y so it visually matches the world layoud. Unvisited cells display as --
- The heatmap PNG rendered with matplotlib. Unvisited cells are displayed as light gray and each visited cell is annotated with its numerical probability

Experimentation and Results

5.1 Parameter Tuning

The K value started with value 10, and I observed that even cells in clearly slippery regions retained mid-range probabilities. Increasing to K=20 produced the desired separation. Increasing the K value above 20 made the function too sensitive to single-observation noise in cells that the robot only briefly passed through.

5.2 Coverage Run

The coverage run completed all 20 waypoints successfully:

```
Quadrant tour complete! reached=20 timeout=0 blocked=0
```

A single rotate_to: timeout warning appeared mid-run during a heading correction, but the subsequent drive_to recovered and reached the waypoint. Total runtime was approximately 140 seconds of simulated time. 47 of the 182 total cells received at least one slip observation. The visited cells form a connected path that touches all four quadrants.

5.3 Final Output Matrix

```
================================================================
TRAVERSABILITY MAP  P(Traversable=True) per 0.5m cell
Grid: 13 rows (y) x 14 cols (x)
================================================================
y[+1.0,+1.5]    --    --  0.802 0.070 0.230   --  0.587 0.792 0.921 0.351 0.456 0.381   --    --
y[+0.5,+1.0]    --    --  0.132 0.092 0.039 0.112 0.429   --    --    --  0.926 0.194   --    --
y[+0.0,+0.5]    --    --    --    --  0.044   --  0.844 0.052 0.238 0.901 0.933   --    --    --
y[-0.5,+0.0]    --    --    --    --  0.092   --    --    --    --    --    --    --    --    --
y[-1.0,-0.5]    --    --    --    --  0.120   --    --    --    --    --    --    --    --    --
y[-1.5,-1.0]    --    --    --    --  0.329   --    --    --    --    --    --    --    --    --
y[-2.0,-1.5]    --    --    --    --  0.985   --    --    --    --    --    --    --    --    --
y[-2.5,-2.0]    --    --    --    --  0.516   --    --    --    --    --    --    --    --    --
y[-3.0,-2.5]    --    --  0.212 0.983 0.376   --    --    --  0.121 0.878 0.976 0.284   --    --
y[-3.5,-3.0]    --    --  0.990   --    --  0.985 0.987 0.944 0.968   --    --  0.972   --    --
y[-4.0,-3.5]    --    --  0.397 0.329 0.645 0.993   --    --    --  0.557 0.910 0.323   --    --
y[-4.5,-4.0]    --    --    --    --    --    --    --    --    --    --    --    --    --    --
y[-5.0,-4.5]    --    --    --    --    --    --    --    --    --    --    --    --    --    --
================================================================
Cells visited: 47/182  (25.8%)
```

5.4 Robot Navigation Output

```
hostuser@0c89483f5318:/mnt/private/ArtificialIntelligence/FinalProject$ python3 coverage_mover.py
[INFO] [1777598808.638374, 990.792000]: Coverage plan: 20 waypoints across 4 quadrants
[INFO] [1777598808.649236, 990.802000]: Waiting for odometry...
[INFO] [1777598810.769890, 992.823000]: Starting quadrant tour
[INFO] [1777598810.775068, 992.824000]: Waypoint 1/20: (1.20, 0.50)  robot at (-0.00, 0.00)
[INFO] [1777598818.481675, 1000.035000]:   -> reached
[INFO] [1777598818.489609, 1000.044000]: Waypoint 2/20: (2.50, 0.50)  robot at (0.82, 0.48)
[INFO] [1777598823.304799, 1004.535000]:   -> reached
[INFO] [1777598823.313478, 1004.541000]: Waypoint 3/20: (2.50, 1.30)  robot at (2.11, 0.50)
[INFO] [1777598826.856695, 1007.835000]:   -> reached
[INFO] [1777598826.865384, 1007.839000]: Waypoint 4/20: (1.20, 1.30)  robot at (2.34, 0.94)
[INFO] [1777598832.991817, 1013.487000]:   -> reached
[INFO] [1777598832.996765, 1013.487000]: Waypoint 5/20: (1.80, 0.90)  robot at (1.58, 1.20)
[INFO] [1777598833.005723, 1013.504000]:   -> reached
[INFO] [1777598833.011186, 1013.511000]: Waypoint 6/20: (-1.20, 0.90)  robot at (1.57, 1.20)
[INFO] [1777598843.108615, 1022.885000]:   -> reached
[INFO] [1777598843.114414, 1022.889000]: Waypoint 7/20: (-2.50, 0.90)  robot at (-0.80, 0.91)
[INFO] [1777598852.088532, 1031.235000]:   -> reached
[INFO] [1777598852.094163, 1031.241000]: Waypoint 8/20: (-2.50, 1.30)  robot at (-2.11, 0.91)
[INFO] [1777598855.889741, 1034.685000]:   -> reached
[INFO] [1777598855.896606, 1034.695000]: Waypoint 9/20: (-1.20, 1.30)  robot at (-2.40, 0.95)
[INFO] [1777598867.536693, 1045.485000]:   -> reached
[INFO] [1777598867.541899, 1045.492000]: Waypoint 10/20: (-1.80, 0.50)  robot at (-1.54, 1.10)
[WARN] [1777598873.980761, 1051.535000]: rotate_to: timeout
[INFO] [1777598877.577133, 1054.885000]:   -> reached
[INFO] [1777598877.609690, 1054.916000]: Waypoint 11/20: (-1.20, -2.80)  robot at (-1.49, 0.74)
[INFO] [1777598897.057578, 1073.085000]:   -> reached
[INFO] [1777598897.066505, 1073.092000]: Waypoint 12/20: (-2.50, -2.80)  robot at (-1.15, -2.42)
[INFO] [1777598902.574127, 1078.286000]:   -> reached
[INFO] [1777598902.605986, 1078.317000]: Waypoint 13/20: (-2.50, -4.00)  robot at (-2.13, -2.69)
[INFO] [1777598908.097423, 1083.485000]:   -> reached
[INFO] [1777598908.108098, 1083.496000]: Waypoint 14/20: (-1.20, -4.00)  robot at (-2.40, -3.63)
[INFO] [1777598913.442778, 1088.485000]:   -> reached
[INFO] [1777598913.448632, 1088.492000]: Waypoint 15/20: (-1.80, -3.40)  robot at (-1.56, -3.88)
[INFO] [1777598917.547246, 1092.235000]:   -> reached
[INFO] [1777598917.551980, 1092.237000]: Waypoint 16/20: (1.20, -2.80)  robot at (-1.62, -3.75)
[INFO] [1777598929.641317, 1103.585000]:   -> reached
[INFO] [1777598929.646349, 1103.590000]: Waypoint 17/20: (2.50, -2.80)  robot at (0.82, -2.93)
[INFO] [1777598935.180739, 1108.735000]:   -> reached
[INFO] [1777598935.187750, 1108.745000]: Waypoint 18/20: (2.50, -4.00)  robot at (2.11, -2.85)
[INFO] [1777598940.360281, 1113.535000]:   -> reached
[INFO] [1777598940.365028, 1113.540000]: Waypoint 19/20: (1.20, -4.00)  robot at (2.40, -3.63)
[INFO] [1777598946.014235, 1118.785000]:   -> reached
[INFO] [1777598946.020150, 1118.790000]: Waypoint 20/20: (1.80, -3.40)  robot at (1.58, -3.90)
[INFO] [1777598950.102936, 1122.585000]:   -> reached
[INFO] [1777598950.108916, 1122.591000]: Quadrant tour complete! reached=20  timeout=0  blocked=0
[INFO] [1777598950.113314, 1122.594000]: Shutting down coverage mover
[INFO] [1777598951.062668, 1122.914000]: Shutting down coverage mover
```

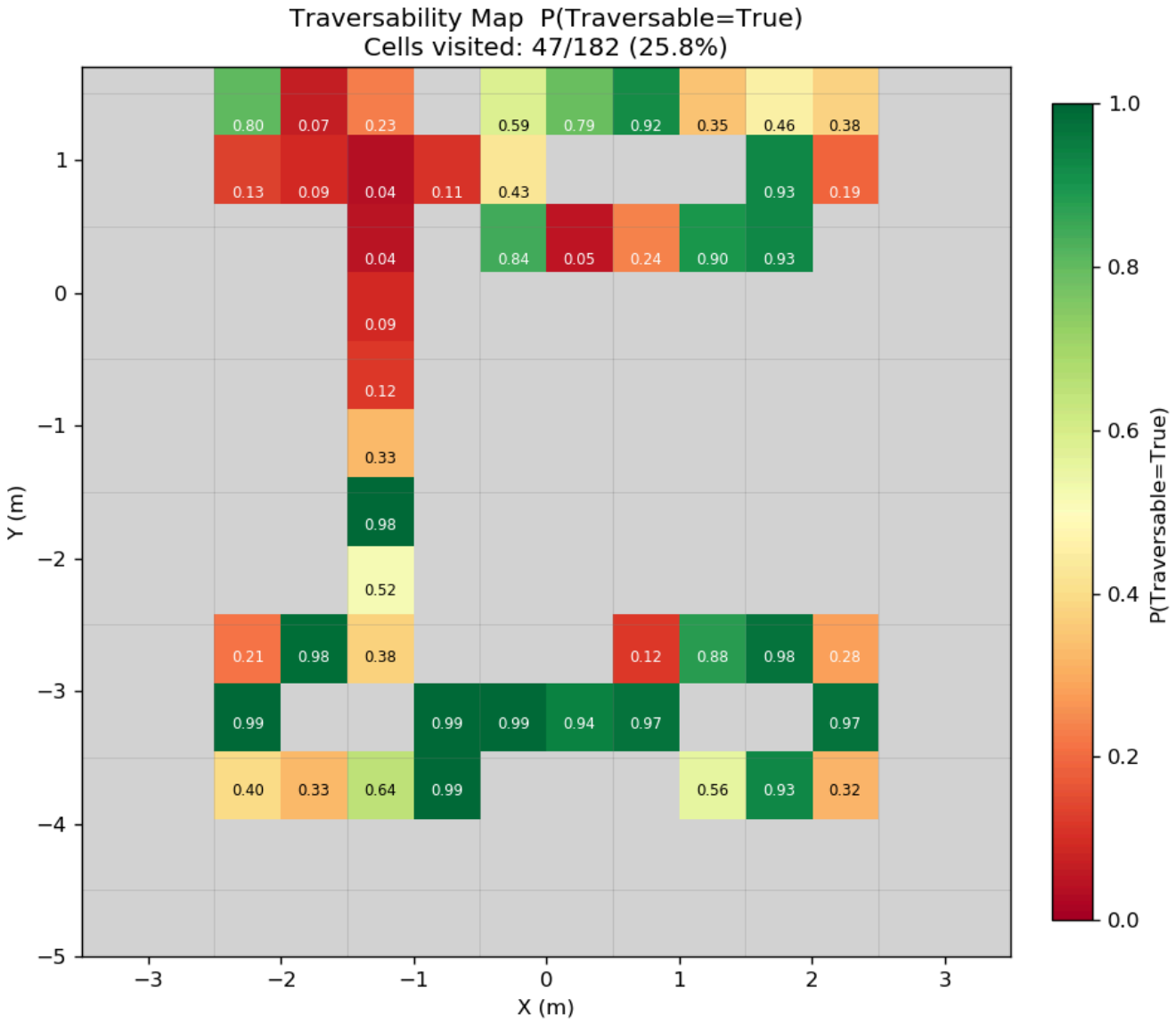
5.5 Results Interpreted

The map clearly separates the four colored regions:

- The purple quadrant posterior probabilities cluster between 0.04 and 0.23 - identified as slippery
- The green quadrant has mostly high probabilities between 0.79 and 0.93 - identified as a gripping terrain
- The teal quadrant has strong probabilities, all at least .98 - identified as the most traversable region

- The pink region had mixed values clustered between 0.28 and 0.97 - identified as moderately slippery. There is high slip on heading transitions and low slip when driving straight
  The narrow vertical band connecting the upper and lower halves is the path the robot took between quadrants. The values along this path are influenced by the turns the robot makes during waypoint transitions which can inflate slip reading.

5.6 Heatmap Output



6. Discussion

6.1 One visit per cell

This, unfortunately, is not a sufficient setup in general. Transient noise dominates a single slip observation within a cell. The robot may be accelerating or correcting its heading the moment the sample is taken, which inflates the slip signal of the underlying terrain. Several cells

illustrate this, such as those in the first quadrant that jump from 0.19 to 0.93, even though they are likely to have the same slippery terrain. Multiple visits, ideally during straight-line driving, create the best opportunity for a high-confidence estimate. However, due to the environment setup itself, this would be difficult to achieve with the unpredictable terrain. A wandering robot, despite how directed it might be programmed, may take a very long time to cover every cell more than once.

6.2 What I would change

The clearest improvement would be to specifically observe slip on commanded linear velocity and ignore it during times of acceleration in the robot's angle. This would eliminate the noisy values as the robot moves and move the values to more confident extremes.

7. Conclusion

The implementation implements the DBN structure, with Traversable as a hidden Boolean variable observed indirectly through WheelSlip, and with the prior of each cell initialized to 0.5. The state estimation algorithm is a Bayesian filter under a static-terrain transition model implemented via per-cell mean slip and a parametric likelihood. The robot motion is a 20-waypoint quadrant tour completed without timeouts or blocks, covering about 25.8% of the grid. The final output matrix correctly separates the four colored quadrants, with high confidence estimates in cells visited during straight-line driving and lower confidence estimates in cells that experienced adverse robot movements.